GUIA VISUAL 051

DO COMANDO AO SCRIPT

*Seis comandos toda manhã, sempre na mesma ordem. Enquanto a sequência só existe nos dedos de quem digita, ela é um **hábito** — não um procedimento. O script é o terminal gravado.*

051
GUIAS LINUX

DO ENTER AO PROCESSO — O QUE ACONTECE EM `./ROTINA.SH`

1 · VOCÊ
`./rotina.sh`
digita e tecla Enter.

→

2 · SHELL
`fork() + execve()`
cria o filho e pede ao kernel para executar o arquivo.

→

3 · KERNEL
`#!`
lê os **dois primeiros bytes**. Não é binário: é *shebang*.

→

4 · KERNEL
`/bin/bash`
carrega o interpretador da linha e lhe entrega o arquivo.

→

5 · BASH
`linha a linha`
lê *enquanto* executa — não valida o arquivo antes.

O passo 3 é a razão de o bit `x` ser obrigatório: sem ele o `execve()` falha **antes** de qualquer leitura. Sem *shebang* algum, o kernel devolve `Exec format error` e o Bash interpreta por cortesia histórica — cortesia que o `cron` e o `systemd` não têm. E, como o passo 5 lê por posição de byte, **erro de sintaxe na linha 50 não impede as linhas 1 a 49 de rodarem**: valide com `bash -n` antes da primeira execução.

EXECUTAR × `SOURCE` — A DIFERENÇA QUE EXPLICA O `.BASHRC`

./SCRIPT.SH

- ✓ **Processo novo.** O filho recebe uma *cópia* do ambiente; ao terminar, a cópia é descartada.
- ✓ Exige bit `x` e *shebang*. `bash script.sh` dispensa os dois.
- ✓ `cd`, `export` e funções **não sobrevivem**: o shell que chamou fica como estava.
- ✓ Depurar: `bash -x script.sh` mostra cada linha expandida.

SOURCE SCRIPT.SH

- ✓ **Mesmo processo.** As linhas são lidas como se tivessem sido digitadas — `$$` não muda.
- ✓ Forma POSIX: `. script.sh`. Não exige bit `x`.
- ✓ Altera a sessão em curso: PWD, variáveis e funções passam a valer.
- ✓ É o mecanismo do `.bashrc` e do `/etc/profile.d/`. Por isso `bash ~/.bashrc` "não faz nada".

O *SHEBANG* CERTO E OS TRÊS ERROS CLÁSSICOS

PRIMEIRA LINHA	O QUE SIGNIFICA
`#!/usr/bin/env bash`	Procura o `bash` no `PATH`. **Padrão portável** — roda em Debian, macOS, BSD.
`#!/bin/bash`	Caminho fixo e absoluto. Use quando você controla o sistema e quer garantia de qual binário roda.
`#!/bin/sh`	No Debian/Kali é o **dash**: sem `[[ ]]`, arrays ou `local`. Em RHEL é o Bash em modo POSIX — o mesmo arquivo funciona lá e falha aqui.

`Permission denied`
Achou o arquivo, falta o bit `x`. → `chmod +x`

`command not found`
O arquivo está fora do `PATH`. → `./` na frente, ou instale em `~/bin`.

`/usr/bin/env^M: bad interpreter`
Editado no Windows: fim de linha CRLF. → `dos2unix` ou `sed -i 's/\r$//'`

Nunca acrescente `.` ao `PATH`. O `PATH="$PATH:."` no `.bashrc` troca segurança por conveniência: basta plantar um arquivo chamado `ls`, `git` ou `sudo` num diretório compartilhado para que o próximo comando digitado ali execute código alheio. A solução correta é `mkdir -p ~/bin` — o `.profile` do Debian/Kali já o acrescenta ao `PATH` se ele existir. E **nunca edite um script longo enquanto ele roda**: o Bash lê por posição de byte e continuará a partir do deslocamento antigo, agora no meio de outra linha. Edite uma cópia e troque com `mv`.